Cours d’algorithmique

Objectif et plan du cours

- **Objectif:**
  - Apprendre les concepts de base de l'algorithmique et de la programmation
  - Etre capable de mettre en œuvre ces concepts pour analyser des problèmes simples et écrire les programmes correspondants
- **Plan:**
  - Généralités (matériel d'un ordinateur, systèmes d'exploitation, langages de programmation, …)
  - Algorithmique (affectation, instructions conditionnelles, instructions itératives, fonctions, procédures, …)

Système Informatique?

- Techniques du traitement **automatique** de l'**information** au moyen des ordinateurs

  Système informatique = ordinateur + périphériques

- Éléments d'un système informatique

Qu'est-ce qu'un programme d'ordinateur?

- Allumez un ordinateur, vous n'en tirerez rien!!
- Pour le faire marcher il faut lui fournir un programme

  Ordinateur = matériel + programme(s)
- Un programme est une suite d'instructions d'ordinateur
- Une instruction est un **ordre** compris par l'ordinateur et qui lui fait exécuté une **action**, c-à-d une modification de son environnement

Les catégories d’ordres

- les ordinateurs, quels qu’ils soient, ne sont fondamentalement capables de comprendre que quatre catégories d'ordres (en programmation, on n'emploiera pas le terme d'ordre, mais plutôt celui d'instructions). Ces quatre familles d'instructions sont :
  - l’affectation de variables
  - la lecture / écriture
  - les tests
  - les boucles

Actions d'un ordinateur : Exemple

- Attendre qu'un nombre soit tapé au clavier
- Sortir à l'écran le nombre entré
- Attendre qu'un nombre soit tapé au clavier
- Sortir à l'écran le nombre entré
- Additionner les deux nombres entrés
- Sortir à l'écran le résultat de l'addition

Ces lignes forment un programme d'ordinateur

Qu'est ce qu'un système d'exploitation?

- Ensemble de programmes qui gèrent le matériel et contrôlent les applications
  - Gestion des périphériques (affichage à l'écran, lecture du clavier, pilotage d'une imprimante, …)
  - Gestion des utilisateurs et de leurs données (comptes, partage des ressources, gestion des fichiers et répertoires, …)
  - Interface avec l'utilisateur (textuelle ou graphique): Interprétation des commandes
  - Contrôle des programmes (découpage en taches, partage du temps processeur, …)

Langages informatiques

- Un langage informatique est un outil permettant de donner des ordres (**instructions**) à la machine
  - A chaque instruction correspond une action du processeur
- Intérêt : écrire des **programmes** (suite consécutive d'instructions) déstinés à effectuer une tache donnée
  - Exemple: un programme de gestion de comptes bancaires
- Contrainte: être compréhensible par la machine

Langage machine

- Langage **binaire**: l'information est exprimée et manipulée sous forme d'une suite de bits

- Un **bit** (*binary digit*) = 0 ou 1 (2 états électriques)

  - Le code **ASCII** (*American Standard Code for Information Interchange*) donne les correspondances entre les caractères alphanumériques et leurs représentation binaire, Ex. A= 01000001, ?=00111111

$2^8 = 256$

- Les opérations logiques et arithmétiques de base (addition, multiplication, …) sont effectuées en binaire

L'assembleur

- Problème: le langage machine est difficile à comprendre par l'humain

- Idée: trouver un langage compréhensible par l'homme qui sera ensuite converti en langage machine
  - **Assembleur** (1er langage): exprimer les instructions élémentaires de façon symbolique

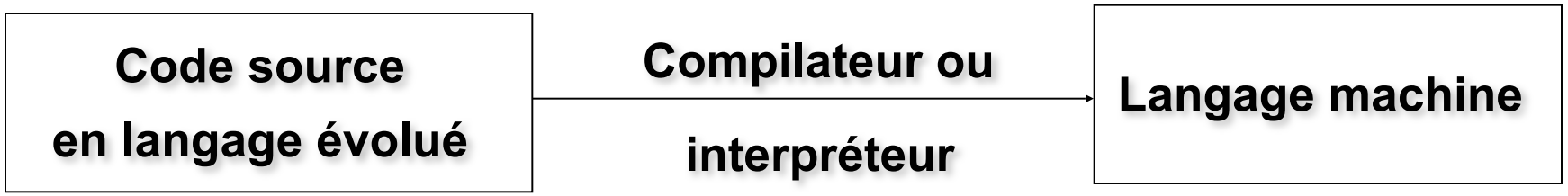
  - +: déjà plus accessible que le langage machine
  - -: dépend du type de la machine (n'est pas ***portable***)
  - -: pas assez efficace pour développer des applications complexes

Apparition des langages évolués

$\Rightarrow$

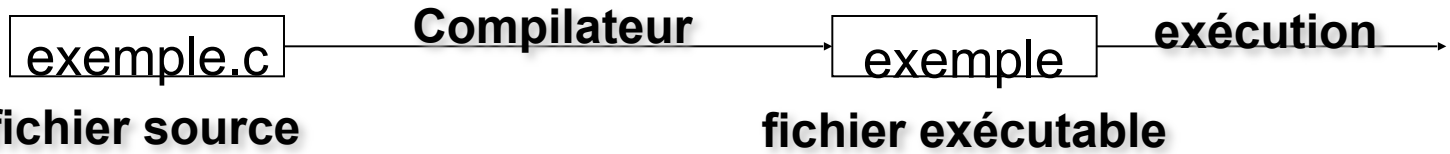
Langages haut niveau

- Intérêts multiples pour le haut niveau:
  - proche du langage humain «anglais» (compréhensible)
  - permet une plus grande portabilité (indépendant du matériel)
  - Manipulation de données et d'expressions complexes (réels, objets, a*b/c, …)
- Nécessité d'un traducteur (compilateur/interpréteur), exécution plus ou moins lente selon le traducteur



Compilateur/interpréteur

- <u>Compilateur:</u> traduire le programme entier une fois pour toutes



  - + plus rapide à l'exécution
  - + sécurité du code source
  - - il faut recompiler à chaque modification

- <u>Interpréteur:</u> traduire au fur et à mesure les instructions

- programme à chaque exécution
  - + exécution instantanée appréciable pour les débutants
  - - exécution lente par rapport à la compilation

Etapes de réalisation d'un programme

Enoncé du problème

↓ *Spécification*

Cahier des charges

↓ *Analyse*

Algorithme

↓ *Traduction en langage*

Programme source

↓ *Compilation*

Programme exécutable

↓ *Tests et modifications*

Version finale et résultats

La réalisation de programmes passe par l'écriture d'algorithmes
⇒ D'où l'intérêt de l'**Algorithmique**

Pourquoi apprendre l'algorithmique pour apprendre à programmer ?

- Un algorithme est une description complète et détaillée des actions à effectuer et de leur séquencement pour arriver à un résultat donné
  - Intérêt: séparation analyse/codage (pas de préoccupation de syntaxe) l'algorithmique exprime les instructions résolvant un problème donné indépendamment des particularités de tel ou tel langage.
  - Qualités: **exact** (fournit le résultat souhaité), **efficace** (temps d'exécution, mémoire occupée), **clair** (compréhensible), **général** (traite le plus grand nombre de cas possibles), …
- Pour prendre une image, si un programme était une dissertation, l'algorithmique serait le plan, une fois mis de côté la rédaction et l'orthographe. Mieux faire d'abord le plan et rédiger ensuite que l'inverse…

Représentation d'un algorithme

Historiquement, deux façons pour représenter un algorithme:

- **L'Organigramme:** représentation graphique avec des symboles (carrés, losanges, etc.)
  - offre une vue d'ensemble de l'algorithme
  - représentation quasiment abandonnée aujourd'hui

- **Le pseudo-code:** représentation textuelle avec une série de conventions ressemblant à un langage de programmation (sans les problèmes de syntaxe)
  - plus pratique pour écrire un algorithme
  - représentation largement utilisée

Algorithmique

Notions et instructions de base

Les catégories d’ordres

- les ordinateurs, quels qu’ils soient, ne sont fondamentalement capables de comprendre que quatre catégories d'ordres (en programmation, on n'emploiera pas le terme d'ordre, mais plutôt celui d'instructions). Ces quatre familles d'instructions sont :
  - Les variables et leurs affectation
  - la lecture / écriture
  - les tests
  - les boucles

Notion de variable

- Dans les langages de programmation une **variable** sert à stocker la valeur d'une donnée

- Une variable désigne en fait un emplacement mémoire dont le contenu peut changer au cours d'un programme (d'où le nom variable)

- Règle : Les variables doivent être **déclarées** avant d'être utilisées, elle doivent être caractérisées par :
  - un nom (**Identificateur**)
  - un **type** (entier, réel, caractère, chaîne de caractères, …)

Choix des identificateurs (1)

Le choix des noms de variables est soumis à quelques règles qui varient selon le langage, mais en général:

- Un nom doit commencer par une lettre alphabétique
  exemple valide: A1 **exemple invalide: 1A**
- doit être constitué uniquement de lettres, de chiffres et du soulignement _ (Eviter les caractères de ponctuation et les espaces)
  valides: SMI2007, SMI_2007 **invalides: SMI 2007, SMI-2007, SMI;2007**
- doit être différent des mots réservés du langage (par exemple en Java: **int, float, else, switch, case, default, for, main, return**, …)
- La longueur du nom doit être inférieure à la taille maximale spécifiée par le langage utilisé

Choix des identificateurs (2)

Conseil: pour la lisibilité du code choisir des noms significatifs qui décrivent les données manipulées

exemples: TotalVentes2006, Prix_TTC, Prix_HT

Remarque: en pseudo-code algorithmique, on va respecter les règles citées, même si on est libre dans la syntaxe

Types des variables

Le type d'une variable détermine l'ensemble des valeurs qu'elle peut prendre, les types offerts par la plus part des langages sont:

- Type numérique (entier ou réel)
  - **Byte** (codé sur 1octet): de 0 à 255
  - **Entier court** (codé sur 2 octets) : -32 768 à 32 767
  - **Entier long** (codé sur 4 ou 8 octets)
  - **Réel simple précision** (codé sur 4 octets)
  - **Réel double précision** (codé sur 8 octets)
- Type logique ou booléen: deux valeurs VRAI ou FAUX
- Type caractère: lettres majuscules, minuscules, chiffres, symboles, …

  exemples: 'A', 'a', '1', '?', …
- Type chaîne de caractère: toute suite de caractères,

  exemples: " Nom, Prénom", "code postale: 1000", …

Déclaration des variables

- Rappel: toute variable utilisée dans un programme doit avoir fait l'objet d'une déclaration préalable
- En pseudo-code, on va adopter la forme suivante pour la déclaration de variables

Variables liste d'identificateurs : type

- Exemple:

Variables i, j,k : entier
x, y : réel
OK: booléen
ch1, ch2 : chaîne de caractères

- Remarque: pour le type numérique on va se limiter aux entiers et réels sans considérer les sous types

L'instruction d'affectation

- **l'affectation** consiste à attribuer une valeur à une variable (ça consiste en fait à remplir où à modifier le contenu d'une zone mémoire)

- En pseudo-code, l'affectation se note avec le signe ←

  Var← e : attribue la valeur de e à la variable Var

  - e peut être une valeur, une autre variable ou une expression
  - Var et e doivent être de même type ou de types compatibles
  - l'affectation ne modifie que ce qui est à gauche de la flèche

- **Ex valides:** **i ←1** **j ←i** **k ←i+j**
  x ←10.3 **OK ←FAUX** **ch1 ←"SMI"**
  ch2 ←ch1 **x ←4** **x ←j**

  (voir la déclaration des variables dans le transparent précédent)

- **non valides:** **i ←10.3** **OK ←"SMI"** **j ←x**

Quelques remarques

- Beaucoup de langages de programmation (C/C++, Java, …) utilisent le signe égal = pour l'affectation ←. Attention aux confusions:
  - l'affectation n'est pas commutative : A=B est différente de B=A
  - l'affectation est différente d'une équation mathématique :
    - A=A+1 a un sens en langages de programmation
    - A+1=2 n'est pas possible en langages de programmation et n'est pas équivalente à A=1
- Certains langages donnent des valeurs par défaut aux variables déclarées. Pour éviter tout problème il est préférable **d'initialiser les variables** déclarées

Exercices simples sur l'affectation (1)

Donnez les valeurs des variables A, B et C après exécution des instructions suivantes ?

Variables A, B, C: **Entier**
Début
A ← 3
B ← 7
A ← B……. B=7; A=7
B ← A+5……A= 7, B=12
C ← A + B……. A= 7, B= 12 , C=19
C ← B – A……; C=5
Fin

Les valeurs sont: A=7; B=12; C=5

Exercices simples sur l'affectation (2)

Donnez les valeurs des variables A et B après exécution des instructions suivantes ?

Variables A, B,C : **Entier**
Début
A ← 1
B ← 2
C ← A…..C=1
A ← B….. A=2
B ← C…….B=1
Fin
B=1; A=2
Les deux dernières instructions permettent-elles d'échanger les valeurs de A et B ?

Expressions et opérateurs

- Une **expression** peut être une valeur, une variable ou une opération constituée de variables reliées par des **opérateurs**
  exemples: 1, b, a*2, a+ 3*b-c, …

- L'évaluation de l'expression fournit une valeur unique qui est le résultat de l'opération

- Les **opérateurs** dépendent du type de l'opération, ils peuvent être :
  - **des opérateurs arithmétiques:** **+, -, *, /, % (modulo), ^ (puissance)**
  - **des opérateurs logiques:** **NON, OU, ET**
  - **des opérateurs relationnels**: **=, ≠, <, >, <=, >=**
  - **des opérateurs sur les chaînes:** **&** (**concaténation)**

- Une expression est évaluée de gauche à droite mais en tenant compte de **priorités**

Priorité des opérateurs

- Pour les opérateurs arithmétiques donnés ci-dessus, l'ordre de priorité est le suivant (du plus prioritaire au moins prioritaire) :
  - ^ : (élévation à la puissance)
  - * , / (multiplication, division)
  - % (modulo)
  - + , - (addition, soustraction)

exemple: 2 + 3 * 7 vaut 23

- En cas de besoin (ou de doute), on utilise les parenthèses pour indiquer les opérations à effectuer en priorité

exemple: (2 + 3) * 7 vaut 35

Les instructions d'entrées-sorties: lecture et écriture (1)

- Les instructions de lecture et d'écriture permettent à la machine de communiquer avec l'utilisateur
- La **lecture** permet d'entrer des <u>donnés</u> à partir du clavier
  - En pseudo-code, on note: **lire (var)**

    la machine met la valeur entrée au clavier dans la zone mémoire nommée var

  - <u>Remarque:</u> Le programme s'arrête lorsqu'il rencontre une instruction Lire et ne se poursuit qu'après la frappe d'une valeur au clavier et de la touche Entrée

Les instructions d'entrées-sorties: lecture et écriture (2)

- **L'écriture** permet d'afficher des <u>résultats</u> à l'écran (ou de les écrire dans un fichier)

  - En pseudo-code, on note: **écrire (A)**

    la machine affiche le contenu de la zone mémoire var

    ecrire(“mon premier algo”)

  - <u>Conseil:</u> Avant de lire une variable, il est fortement conseillé d’écrire des messages à l’écran, afin de prévenir l’utilisateur de ce qu’il doit frapper

Exemple (lecture et écriture)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui calcule et affiche le double de ce nombre

Algorithme Calcul_double

variables A, B : **entier**

Début

écrire("entrer la valeur de A ")

lire(A)

B ← 2*A

écrire("le double de ", A, "est :", B)

Fin

Exercice (lecture et écriture)

Ecrire un algorithme qui vous demande de saisir votre nom puis votre prénom et qui affiche ensuite votre nom complet

Algorithme AffichageNomComplet

variables Nom, Prenom, Nom_Complet : **chaîne de caractères**

Début

écrire("entrez votre nom")

lire(Nom)

écrire("entrez votre prénom")

lire(Prenom)

Nom_Complet ← Nom & Prenom

écrire("Votre nom complet est : ", Nom_Complet)

Fin

Méthode de construction d'un algorithme simple (1/4)

Exemple :

Écrire un algorithme qui consiste a calculer l'air S d'un cercle selon la formule $S = Pi * R^2$

Rappel : Pi = 3.14159 et R le rayon du cercle

Méthode de construction d'un algorithme simple (2/4)

Méthodologie a suivre :

- constantes : Pi = 3.14159
- Variables : Rayon, Surface
- Types : Rayon, Surface : réel
- Expressions et affectation : Surface := Pi * $(Rayon)^2$
- Structures conditionnelles et les boucles : ------
- Opérations d'entrée-sortie : Lire (Rayon),
  Écrire (Surface)

Méthode de construction d'un algorithme simple (3/4)

Algorithme

Calcul_Aire
Constantes
    Pi = 3,14159
Variables
    Rayon, Surface : **réels**
Début
    lire (Rayon)
    Surface := Pi * $(Rayon)^2$
    écrire (Surface)
Fin

Méthode de construction d'un algorithme simple (3/4)

Programme C

```
#include <stdio.h>
#include <math.h>
Main ( )
{
Float  Pi = 3.14159;
Float rayon, surface;
Scanf (« °/°f », &rayon);
surface = pi*pow (rayon,2);
Printif (« °/°f\n »surface, )
Return 0;
}
```

Algorithmique

Les structures Conditionnelles et les boucles

Besoin a des concepts de ruptures de séquence

Algorithme

Calcul_Aire
Constantes
    Pi = 3,14159
Variables
    Rayon, Surface : **réels**
Début
    lire (Rayon)
    Surface := Pi * $(Rayon)^2$
    écrire (Surface)
Fin

- Rare les algorithme qui peuvent se décrire uniquement par un enchaînement séquentiel d'opération élémentaire

- On a besoin a des concept de rupture de séquence comme les test et les boucles

Ex :

✓ un algorithme qui résout une équation de deuxième degré

✓ un algorithme qui calcule une série numérique

Les structures conditionnelles et les boucles

- Les tests simples : permet de réaliser un choix parmi deux possibilités (Ex :Booléenne : vrais ou faux)
- Les instructions conditionnelles : c'est un concept de tests multiples, permet de comparer un objet à une série de valeurs, et exécuter si la condition est vérifier (Ex : recherche des nombres premier dans une ensemble)
- Les itérations : consiste a exécuté un bloc d'instructions un certain nombre de fois (Ex : calcul d'une suite numérique)
- Les boucles conditionnelles : consiste a exécuté un bloc d'instructions un certain nombre de fois si la condition est vérifier (Ex : On veut afficher le 100 premiers nombres :. Tant que i est plus petit que 100, afficher la valeur de i).

Tests: instructions conditionnelles (1)

- Les instructions conditionnelles servent à n'exécuter une instruction ou une séquence d'instructions que si une condition est vérifiée
- On utilisera la forme suivante: **Si** condition **alors**

  instruction ou suite d'instructions1

  Sinon

  instruction ou suite d'instructions2

  Finsi

  - la condition ne peut être que vraie ou fausse
  - si la condition est vraie, se sont les instructions1 qui seront exécutées
  - si la condition est fausse, se sont les instructions2 qui seront exécutées
  - la condition peut être une condition simple ou une condition composée de plusieurs conditions

Tests: instructions conditionnelles (2)

- La partie Sinon n'est pas obligatoire, quand elle n'existe pas et que la condition est fausse, aucun traitement n'est réalisé
  - On utilisera dans ce cas la forme simplifiée suivante:

Si condition **alors**

instruction ou suite d'instructions1

Finsi

Exemple (Si…Alors…Sinon)

Algorithme AffichageValeurAbsolue (version1)

Variable x : **réel**

Début

Ecrire " Entrez un réel : "

Lire (x)

Si x < 0 **alors**

Ecrire ("la valeur absolue de ", x, "est:",-x)

Sinon

Ecrire ("la valeur absolue de ", x, "est:",x)

Finsi

Fin

Exemple (Si…Alors)

Algorithme AffichageValeurAbsolue (version2)

Variable x,y : **réel**

Début

Ecrire " Entrez un réel : "

Lire (x)

y← x

Si x < 0 **alors**

y ← -x

Finsi

Ecrire ("la valeur absolue de ", x, "est:",y)

Fin

Exercice (tests)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui teste et affiche s'il est divisible par 3

```
Algorithme Divsible_par3
Variable n : entier
Début
        Ecrire " Entrez un entier : "
        Lire (n)
        Si (n%3=0) alors
                Ecrire (n," est divisible par 3")
        Sinon
                Ecrire (n," n'est pas divisible par 3")
        Finsi
Fin
```

Tables de vérité

C1	C2	**C1 ET C2**
VRAI	VRAI	**VRAI**
VRAI	FAUX	**FAUX**
FAUX	VRAI	**FAUX**
FAUX	FAUX	**FAUX**

C1	C2	**C1 OU C2**
VRAI	VRAI	**VRAI**
VRAI	FAUX	**VRAI**
FAUX	VRAI	**VRAI**
FAUX	FAUX	**FAUX**

C1	**NON C1**
VRAI	**FAUX**
FAUX	**VRAI**

Tests imbriqués

- Les tests peuvent avoir un degré quelconque d'imbrications

```
Si condition1 alors
        Si condition2 alors
                instructionsA
            Sinon
                instructionsB
            Finsi
        Sinon
            Si condition3 alors
                instructionsC
            Finsi
        Finsi
```

Tests imbriqués: exemple (version 1)

```
Variable n : entier
Début
        Ecrire ("entrez un nombre : ")
        Lire (n)
        Si n < 0 alors
                Ecrire ("Ce nombre est négatif")
Sinon
        Si n = 0 alors
                        Ecrire ("Ce nombre est nul")
        Sinon
                        Ecrire ("Ce nombre est positif")
        Finsi
        Finsi

Fin
```

Tests imbriqués: exemple (version 2)

```
Variable n : entier
 Début
        Ecrire ("entrez un nombre : ")
        Lire (n)
Si n < 0 alors Ecrire ("Ce nombre est négatif")
        Finsi
Si n = 0 alors Ecrire ("Ce nombre est nul")
Finsi
        Si n > 0 alors      Ecrire ("Ce nombre est positif")
Finsi
Fin
```

Remarque : dans la version 2 on fait trois tests systématiquement alors que dans la version 1, si le nombre est négatif on ne fait qu'un seul test

Conseil : utiliser les tests imbriqués pour limiter le nombre de tests et placer d'abord les conditions les plus probables (minimiser la complexité)

Tests imbriqués: exercice

Le prix de photocopies dans une reprographie varie selon le nombre demandé: 0,5 DH la copie pour un nombre de copies inférieur à 10, 0,4DH pour un nombre compris entre 10 et 20 et 0,3DH au-delà.

Ecrivez un algorithme qui demande à l'utilisateur le nombre de photocopies effectuées, qui calcule et affiche le prix à payer

Tests imbriqués: corrigé de l'exercice

```
Variables copies : entier
          prix : réel
Début
    Ecrire ("Nombre de photocopies : ")
Lire (copies)
    Si copies < 10 Alors
        prix ← copies*0.5
Sinon Si copies < 20
            prix ← copies*0.4
        Sinon
            prix ← copies*0.3
        Finsi
Finsi
    Ecrire (“Le prix à payer est : ”, prix)
Fin
```

Tests imbriqués: Exercice 2

Écrire l'algorithme du traitement qui calcule le discriminant DELTA d'trinome du second degré AX2 + BX + C et qui, en fonction de son signe, calcule la ou les racines réelles du trinome ou afiche, si besoin est qu'il n'ya pas de racine réelle.

Les trois coefficients A, B et C seront saisis au clavier avant traitement.

Tests imbriqués: corrigé de l'exercice 2

```
Variables
        A, B, C, Delta, X1, X2 : réels
  Début
        Lire (A, B, C)
        Delta ← B² – 4 AC

        Si (Delta < 0) Alors
         Ecrire (« le trinome n'a pas de racine réelle »)
        Sinon
                Si (Delta > 0 Alors
                        X1 ← (-B + racine(delta)) / 2A
                        X2 ← (-B - racine(delta)) / 2A

                        Ecrire (« le trinome possède deux racines réelles : », X1, X2)

                Sinon

                        X1 ← (-B ) / 2A
                        Ecrire (« le trinome possède une racine réelle : », X1)
                Finsi
         Finsi

    Fin
```

Algorithmique

Les boucles

Les types de boucle

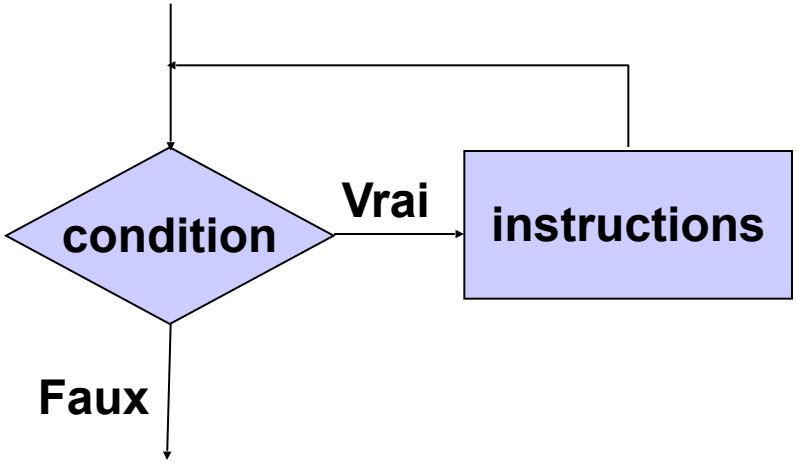
- On distingue 2 types de boucles:
  - Les boucles à compteur ou définie
    - On sait à l'avance combien de fois la boucle devra tourner et une variable (le compteur) compte les répétitions
      - Choisir 10 nombres au hasard. On fera dix fois l'opération choisir un nombre au hasard.
      - Ex : **la boucle Pour**
  - Les boucles à événement ou indéfinie
    - On ne sait pas à l'avance le nombre de fois que la boucle sera exécutée.
      - Ça peut dépendre du nombre de données à traiter.
      - Ça peut dépendre du nombre d'essais que l'usager a effectués.
      - Ex : **la boucle Tanque** et **la boucle jusqu'a**

Les boucles Tant que

TantQue (condition)

instructions

FinTantQue



Boucle Tant que : exemple simple

Un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100

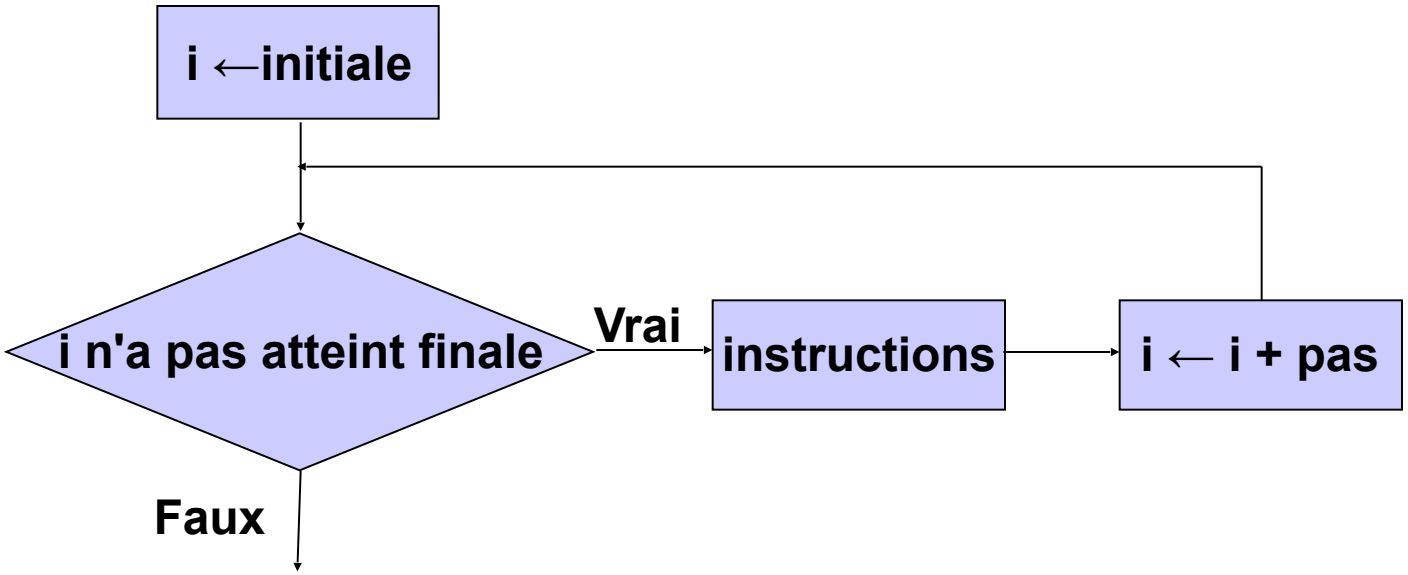
```
Variables som, i : entier
Debut
        i ← 0
        som← 0
        TantQue (som <=100)
                i ←  i+1
                som ←  som+i
        FinTantQue
        Ecrire (" La valeur cherchée est N=  ", i)
Fin
```

Les boucles Pour

Pour compteur **allant de** initiale **à** finale par **pas** valeur du pas

instructions

FinPour



Les boucles Pour

Remarques :

- **Compteur** est une variable de type entier (ou caractère). Elle doit être déclarée

- **Pas** est un entier qui peut être positif ou négatif. **Pas** peut ne pas être mentionné, car par défaut sa valeur est égal à 1. Dans ce cas, le nombre d'itérations est égal à finale - initiale+ 1

- **Initiale et finale** peuvent être des valeurs, des variables définies avant le début de la boucle ou des expressions de même type que compteur

Déroulement des boucles Pour

1) La valeur initiale est affectée à la variable compteur

2) On compare la valeur du compteur et la valeur de finale :

   a) Si la valeur du compteur est > à la valeur finale dans le cas d'un pas positif (ou si compteur est < à finale pour un pas négatif), on sort de la boucle et on continue avec l'instruction qui suit FinPour

   b) Si compteur est <= à finale dans le cas d'un pas positif (ou si compteur est >= à finale pour un pas négatif), instructions seront exécutées

      i. Ensuite, la valeur de compteur est incrémentée de la valeur du pas si pas est positif (ou décrémenté si pas est négatif)

      ii. On recommence l'étape 2 : La comparaison entre compteur et finale est de nouveau effectuée, et ainsi de suite …

Boucle Pour : exemple

Algorithme Plus-Grand-Element: Réécriture de l'algorithme précédent mais avec une boucle ``Pour''

Entrée: *n* entiers S1,…, Sn

Sortie: *grand* contenant le plus grand élément

Algo *plus-grand (S,n)*

grand := S_1;

Pour *i* =1 **à** *n* **Faire**

Si S_i > *grand* **alors** // une plus grande valeur a été trouvée

grand := S_i;

ecrire *(grand)*

Fin *plus-grand;*

Boucle Pour : remarque

- Il faut éviter de modifier la valeur du compteur (et de finale) à l'intérieur de la boucle. En effet, une telle action :
  - perturbe le nombre d'itérations prévu par la boucle Pour
  - rend difficile la lecture de l'algorithme
  - présente le risque d'aboutir à une boucle infinie

Exepmle : **Pour** i **allant de** 1 **à** 5

i ← i -1

écrire(" i = ", i)

Finpour

Lien entre Pour et TantQue

La boucle Pour est un cas particulier de Tant Que (cas où le nombre d'itérations est connu et fixé) . Tout ce qu'on peut écrire avec Pour peut être remplacé avec TantQue (la réciproque est fausse)

Pour compteur **allant de** initiale **à** finale par **pas** valeur du pas

instructions

FinPour

peut être remplacé par :

(cas d'un pas positif)

compteur ← initiale

TantQue compteur <= finale

instructions

compteur ← compteur+pas

FinTantQue

Lien entre Pour et TantQue: exemple

Calcul de **x** à la puissance **n** avec la boucle **Pour** et la boucle **TantQue**

x : un réel non nul

n : entier positif ou nul

Solution avec la boucle Pour

```
Variables x, puiss : réel
                     n, i : entier
Debut
    Ecrire (" Entrez respectivement les valeurs de x et n ")
    Lire (x, n)
    puiss ← 1
    Pour i allant de 1 à n
            puiss← puiss*x
    FinPour
    Ecrire (x, " à la puissance  ", n, "  est égal à  ", puiss)
Fin
```

Solution avec la boucle Tant Que

```
Variables x, puiss : réel
                    n, i : entier
Debut
    Ecrire (" Entrez respectivement les valeurs de x et n ")
    Lire (x, n)
    puiss ← 1, i ← 1
    TantQue (i<=n)
            puiss← puiss*x
            i ← i+1
    FinTantQue
    Ecrire (x, " à la puissance  ", n, "  est égal à  ", puiss)
Fin
```

Algorithme de la fonction factorielle : Exemple

- Écrire deux algorithmes qui calculent pour un entier positif donné **n** la valeur **n!**, un de ces algorithmes doit utilisé la boucle Pour et l'autre la boucle Tanque

<u>Entrée</u> : n de type naturel

<u>Sortie</u> : factoriel (n) = 1*2*3*…..*(n-1)*n

Algorithme de la fonction factorielle

Algorithme / tantque

Calcul factorielle 1
Variables
i, f, n : **Naturel**
Début
i ← 1
f ← 1
tant que (i < n)
i ← i+1
f ← f * i
Fin de tant que
écrire (f)
Fin

Algorithme / Pour

Calcul factorielle 2
Variables
i, f, n : Naturel
Début
f ← 1
pour i variant de 2 à n
f ← f * i
Fin pour
écrire (f)
Fin

Algorithme de la recherche des nombres premiers : Exemple

- Problème: Écrire l'algorithme estPremier, qui a partir d'un entier strictement positif donné, retourne le résultat booléen VRAI ou FAUX selon le nombre est premier ou non.
- Procédure : pour déterminer si un entier *m* est un nombre premier. Il suffit de tester si *m* est divisible par un entier entre *2* et *m/2*
  - *Ex : 1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47. (listes des nombres premier <=50)*

Boucles imbriquées

- Les instructions d'une boucle peuvent être des instructions itératives. Dans ce cas, on aboutit à des **boucles imbriquées**

- **Exemple:**

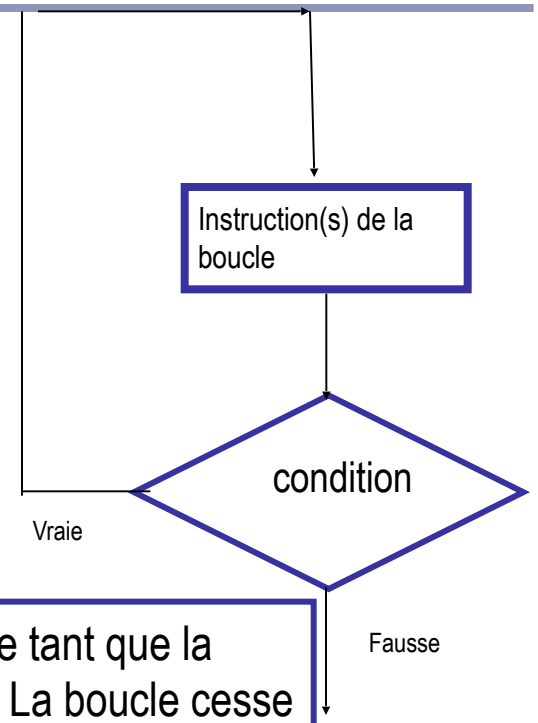
```
Pour i allant de 1 à 5
        Pour j allant de 1 à i
                écrire("O")
        FinPour
        écrire("X")
FinPour
```

Exécution

La boucle Faire…Tant Que

- Faire
  Instruction(s)
  Tant que (condition)
- do
  Instruction;
  while (condition)

```
do
{
      f :=n;
      n--;
} while (n>1);
```



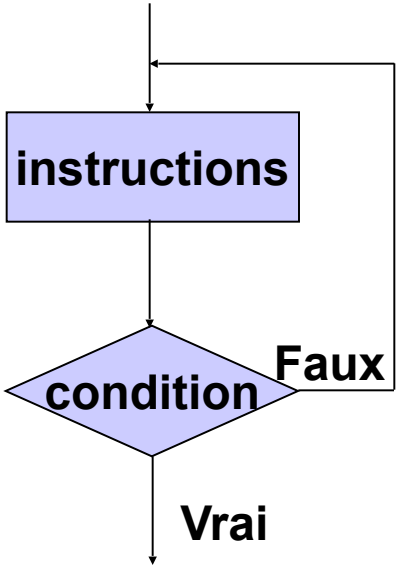
La boucle s'exécute tant que la condition est vraie. La boucle cesse lorque la condition est fausse. À utiliser si l'on veut que la boucle soit exécutée au moins une fois

Les boucles Répéter … jusqu'à …

Répéter

instructions

Jusqu'à condition



- Condition est évaluée après chaque itération
- les instructions entre *Répéter* et *jusqu'à* sont exécutées <u>au moins une fois</u> et leur exécution est répétée jusqu'à ce que condition soit vrai (tant qu'elle est fausse)

Boucle Répéter jusqu'à : exemple

Un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100 **(version avec répéter jusqu'à)**

```
Variables som, i : entier
Debut
        som ← 0
         i ← 0
        Répéter
                i ←  i+1
                som ←  som+i
        Jusqu'à ( som > 100)
        Ecrire (" La valeur cherchée est N=  ", i)
Fin
```

Choix d'un type de boucle

- Si on peut déterminer le nombre d'itérations avant l'exécution de la boucle, il est plus naturel d'utiliser *la boucle Pour*

- S'il n'est pas possible de connaître le nombre d'itérations avant l'exécution de la boucle, on fera appel à l'une des *boucles TantQue ou répéter jusqu'à*

- Pour le choix entre *TantQue* et *jusqu'à :*

  - Si on doit tester la condition de contrôle avant de commencer les instructions de la boucle, on utilisera *TantQue*

  - Si la valeur de la condition de contrôle dépend d'une première exécution des instructions de la boucle, on utilisera *répéter jusqu'à ou faire tanque*

Algorithmique

Les tableaux

Exemple introductif

- Supposons qu'on veut conserver les notes d'une classe de 30 étudiants pour extraire quelques informations. Par exemple : calcul du nombre d'étudiants ayant une note supérieure à 10
- Le seul moyen dont nous disposons actuellement consiste à déclarer 30 variables, par exemple **N1, …, N30**. Après 30 instructions lire, on doit écrire 30 instructions Si pour faire le calcul

nbre ← 0

Si (N1 >10) alors nbre ←nbre+1 FinSi

….

Si (N30>10) alors nbre ←nbre+1 FinSi

c'est lourd à écrire

- Heureusement, les langages de programmation offrent la possibilité de rassembler toutes ces variables dans **une seule structure de donnée** appelée **tableau**

Tableaux

- Un **tableau** est un ensemble d'éléments de même type désignés par un identificateur unique
- Une variable entière nommée **indice** permet d'indiquer la position d'un élément donné au sein du tableau et de déterminer sa valeur
- La **déclaration** d'un tableau s'effectue en précisant le **type** de ses éléments et sa **dimension** (le nombre de ses éléments)
  - <u>En pseudo code :</u>

    variable **tableau** identificateur**[dimension] : type**
  - <u>Exemple :</u>

    variable **tableau** notes**[30] : réel**
- On peut définir des tableaux de tous types : tableaux d'entiers, de réels, de caractères, de booléens, de chaînes de caractères, …

Tableaux : remarques

- L'accès à un élément du tableau se fait au moyen de l'indice. Par exemple, **notes[i]** donne la valeur de l'élément i du tableau notes
- Selon les langages, le premier indice du tableau est soit 0, soit 1. Le plus souvent c'est 0 (c'est ce qu'on va adopter en pseudo-code). Dans ce cas, **notes[i]** désigne l'élément i+1 du tableau notes
- Il est possible de déclarer un tableau sans préciser au départ sa dimension. Cette précision est faite ultérieurement
  - Par exemple, quand on déclare un tableau comme paramètre d'une procédure, on peut ne préciser sa dimension qu'au moment de l'appel
  - En tous cas, un tableau est inutilisable tant qu'on n'a pas précisé le nombre de ses éléments
- Un grand avantage des tableaux est qu'on peut traiter les données qui y sont stockées de façon simple en utilisant des boucles

Tableaux : exemples (1)

- Pour le calcul du nombre d'étudiants ayant une note supérieure à 10 avec les tableaux, on peut écrire :

```
Variables      i ,nbre : entier
               tableau notes[30] : réel
Début
      nbre ← 0
      Pour i allant de 0 à 29
               Si (notes[i]  >10) alors
                     nbre ←nbre+1
               FinSi
      FinPour
      écrire ("le nombre de notes supérieures à 10 est : ", nbre)
Fin
```

Tableaux : saisie et affichage

- Procédures qui permettent de saisir et d'afficher les éléments d'un tableau :

```
Procédure SaisieTab(n : entier par valeur, tableau T : réel par référence )
 variable i: entier
   Pour i allant de 0 à n-1
      écrire ("Saisie de l'élément ",  i + 1)
      lire (T[i] )
   FinPour
Fin Procédure

Procédure AfficheTab(n : entier par valeur, tableau T : réel par valeur )
 variable i: entier
   Pour i allant de 0 à n-1
      écrire ("T[",i, "] =", T[i])
   FinPour
Fin Procédure
```

Tableaux : exemples d'appel

- Algorithme principale où on fait l'appel des procédures SaisieTab et AfficheTab :

Algorithme Tableaux
variable p : entier
    tableau A[10] : réel
Début
    p ← 10
    SaisieTab(p, A)
    AfficheTab(p,A)
Fin

Tableaux à deux dimensions

- Les langages de programmation permettent de déclarer des tableaux dans lesquels les valeurs sont repérées par **deux indices**. Ceci est utile par exemple pour représenter des matrices

- En pseudo code, un tableau à deux dimensions se **déclare** ainsi :

  variable **tableau** identificateur**[dimension1] [dimension2] : type**

  - Exemple : une matrice A de 3 lignes et 4 colonnes dont les éléments sont réels

    variable **tableau** A**[3][4] : réel**

- **A[i][j]** permet d'accéder à l'élément de la matrice qui se trouve à l'intersection de la ligne i et de la colonne j

Exemples : lecture d'une matrice

- Procédure qui permet de saisir les éléments d'une matrice :

```
Procédure SaisieMatrice(n : entier par valeur, m : entier par valeur ,
                        tableau A : réel par référence )
Début
 variables i,j : entier
   Pour i allant de 0 à n-1
        écrire ("saisie de la ligne ",  i + 1)
        Pour j allant de 0 à m-1
            écrire ("Entrez l'élément de la ligne ",  i + 1, " et de la colonne ", j+1)
            lire (A[i][j])
        FinPour
   FinPour
Fin Procédure
```

Exemples : affichage d'une matrice

- Procédure qui permet d'afficher les éléments d'une matrice :

```
Procédure AfficheMatrice(n : entier par valeur, m : entier par valeur
                                    ,tableau A : réel par valeur )
Début
 variables i,j : entier
   Pour i allant de 0 à n-1
       Pour j allant de 0 à m-1
           écrire ("A[",i, "] [",j,"]=", A[i][j])
       FinPour
   FinPour
Fin Procédure
```